

Projeto Prático da C1: Inteligência Artificial

Classificação de Imagens (Gatos vs. Cachorros)

Italo Faria Tomazeli

31 de Março de 2026

1 Descrição do Dataset e Justificativa

Para este projeto, foi selecionado o dataset "Cats vs Dogs"(Gatos vs. Cachorros), disponível publicamente no Kaggle. Trata-se de um problema clássico de classificação binária na área de Visão Computacional.

Como justificativa para a viabilidade técnica e para evitar a sobrecarga de processamento no navegador durante o uso do Teachable Machine, foi extraída uma amostra balanceada contendo 400 imagens no total, divididas em 2 classes bem definidas: 200 imagens de Gatos e 200 imagens de Cachorros.

2 Treinamento no Teachable Machine

O modelo base foi treinado utilizando a interface gráfica do Google Teachable Machine. As imagens foram organizadas nas duas classes e submetidas ao processo de treinamento padrão da plataforma.

Após a finalização do treinamento, testes preliminares foram realizados diretamente na plataforma, demonstrando a capacidade de reconhecimento da Inteligência Artificial. O modelo foi então exportado no formato **TensorFlow.js** (gerando os arquivos `model.json`, `metadata.json` e `weights.bin`).

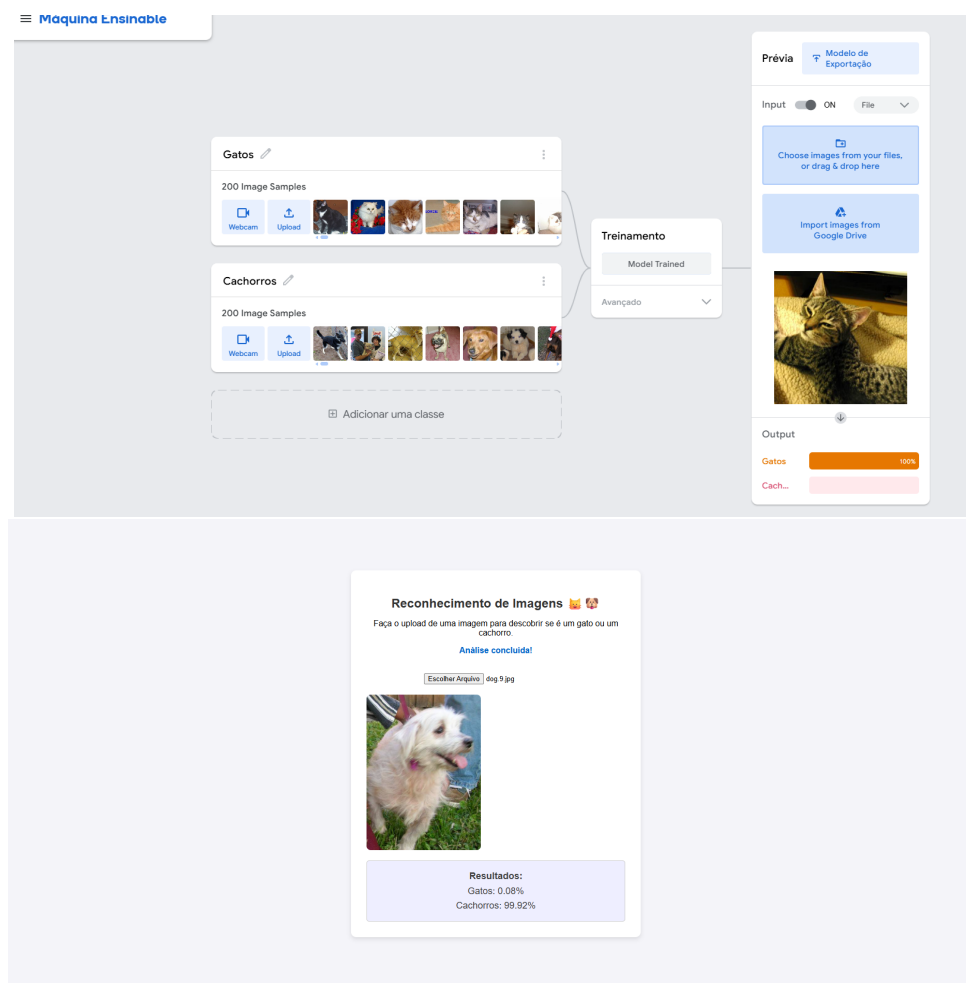


Figura 1: Print da tela de treinamento no Teachable Machine

3 Descrição da Implementação Web

A aplicação web foi desenvolvida utilizando HTML5, CSS3 e JavaScript. O principal requisito foi o carregamento do modelo pré-treinado através da biblioteca `TensorFlow.js` em conjunto com a API do Teachable Machine.

A interface permite que o usuário faça o upload de uma imagem local. Após o carregamento, a imagem é repassada ao modelo preditivo, que calcula as probabilidades. O resultado final (a classe predita e sua porcentagem de confiança) é exibido na página. Para contornar políticas de segurança de CORS (Cross-Origin Resource Sharing) durante o desenvolvimento local, a aplicação foi executada através da extensão Live Server no Visual Studio Code.

4 Resultados da Avaliação do Modelo

Visando uma avaliação rigorosa, foi desenvolvido um script em Python utilizando Google Colab, TensorFlow e Scikit-Learn.

Aplicou-se o **Princípio de Pareto**, dividindo o dataset em 80% para treinamento e 20% para validação/teste. Após o treinamento do modelo com as mesmas classes, as seguintes métricas foram extraídas:

- **Acurácia (Accuracy):** Proporção de acertos totais do modelo.
- **Precisão (Precision):** Capacidade do modelo de não rotular um exemplo negativo como positivo.
- **Revocação (Recall):** Capacidade do modelo de encontrar todas as amostras positivas.
- **F1-Score:** Média harmônica entre Precisão e Revocação.

```

... Found 400 files belonging to 2 classes.
Using 320 files for training.
Found 400 files belonging to 2 classes.
Using 80 files for validation.
/usr/local/lib/python3.12/dist-packages/keras/src/layers/preprocessing/data_layer.py:95: UserWarning: Do not
super().__init__(**kwargs)

Iniciando o treinamento (pode levar alguns segundos)...
Epoch 1/5
10/10 ----- 12s 1s/step - accuracy: 0.4875 - loss: 8.5144 - val_accuracy: 0.5250 - val_loss:
Epoch 2/5
10/10 ----- 20s 1s/step - accuracy: 0.5281 - loss: 2.2641 - val_accuracy: 0.6000 - val_loss:
Epoch 3/5
10/10 ----- 9s 942ms/step - accuracy: 0.6625 - loss: 0.8556 - val_accuracy: 0.6125 - val_loss:
Epoch 4/5
10/10 ----- 10s 1s/step - accuracy: 0.7750 - loss: 0.4986 - val_accuracy: 0.5875 - val_loss:
Epoch 5/5
10/10 ----- 11s 1s/step - accuracy: 0.8313 - loss: 0.3692 - val_accuracy: 0.5875 - val_loss:

--- GERANDO MÉTRICAS PARA O RELATÓRIO ---

Relatório de Classificação (Precision, Recall, F1-Score e Accuracy):
      precision    recall  f1-score   support

   Cachorros      0.65      0.29      0.40        38
     Gatos       0.57      0.86      0.69        42

   accuracy
  macro avg      0.61      0.57      0.54        80
 weighted avg      0.61      0.59      0.55        80

```

Figura 2: Métricas de avaliação geradas via código Python

A Matriz de Confusão gerada detalha os verdadeiros positivos e falsos negativos para cada classe, consolidando a confiabilidade das predições:

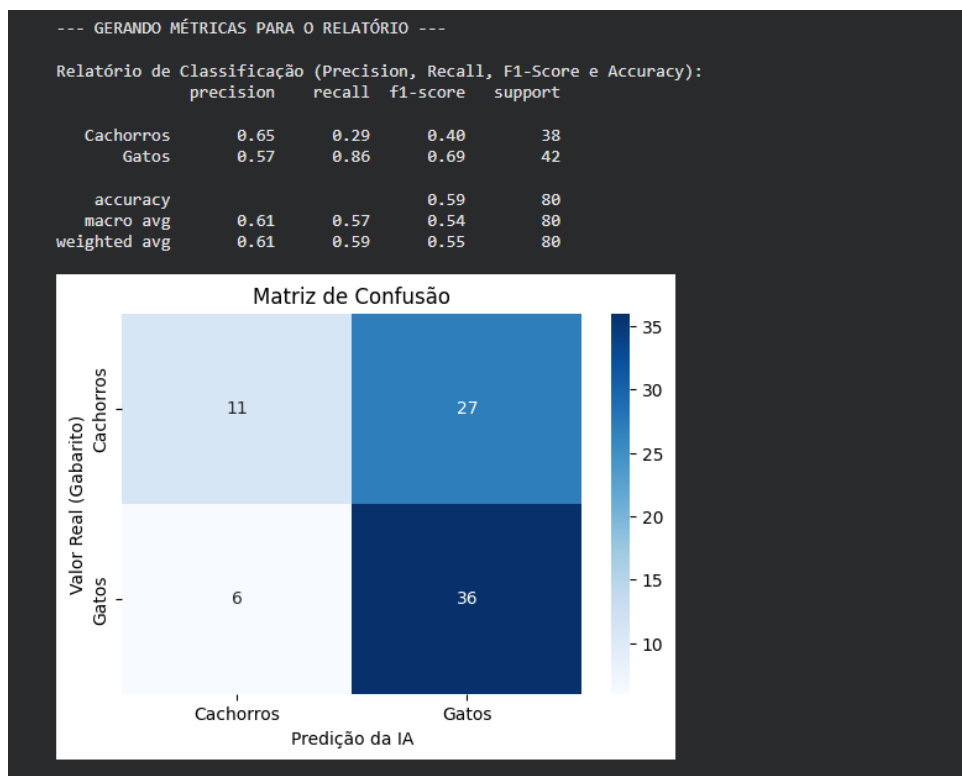


Figura 3: Matriz de Confusão do modelo

5 Conclusões e Possíveis Melhorias

O projeto demonstrou com sucesso o pipeline completo de uma aplicação de Machine Learning: coleta e formatação dos dados, treinamento, deploy em uma interface web e a validação matemática das métricas. O modelo mostrou-se capaz de distinguir as classes propostas.

Como **possíveis melhorias** para iterações futuras, sugere-se:

1. Ampliar o volume do dataset, incluindo imagens com diferentes condições de iluminação e ângulos, aumentando a generalização do modelo.
2. Implementar o recurso de captura de imagem via webcam diretamente na interface web.
3. Realizar um *fine-tuning* (ajuste fino) dos parâmetros no código em Python para reduzir eventuais falsos positivos.